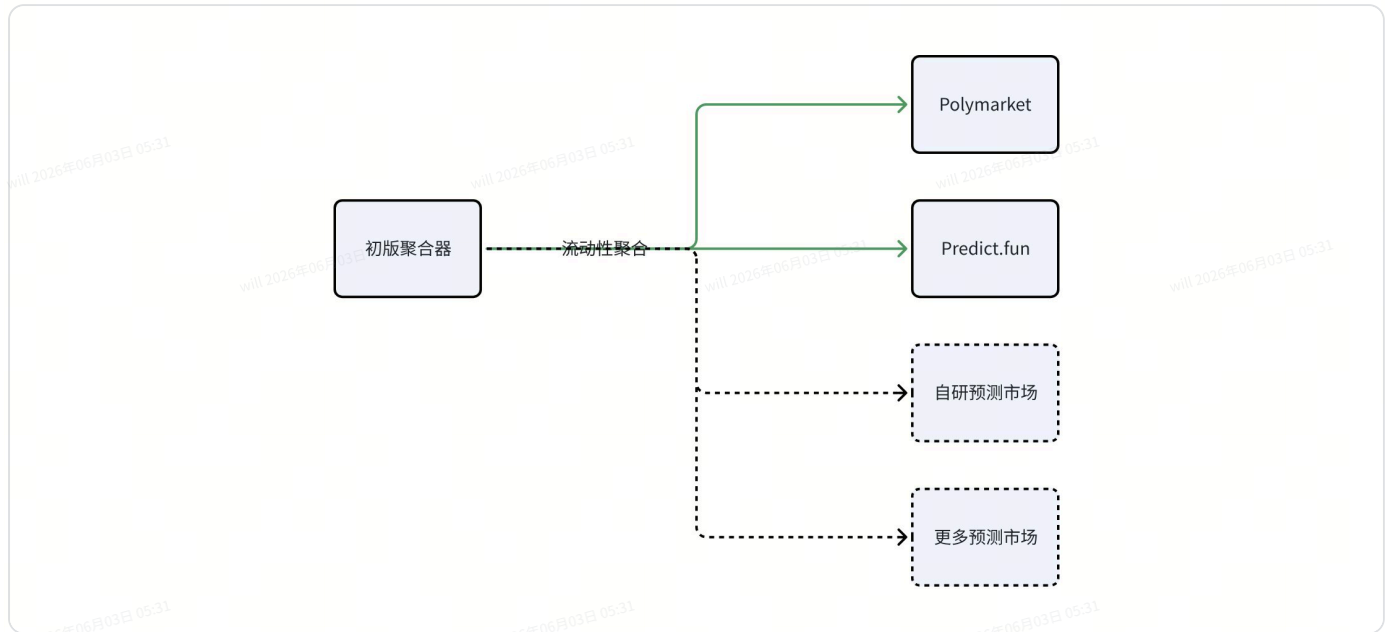


Prediction market聚合器技术方案

0x0 Roadmap



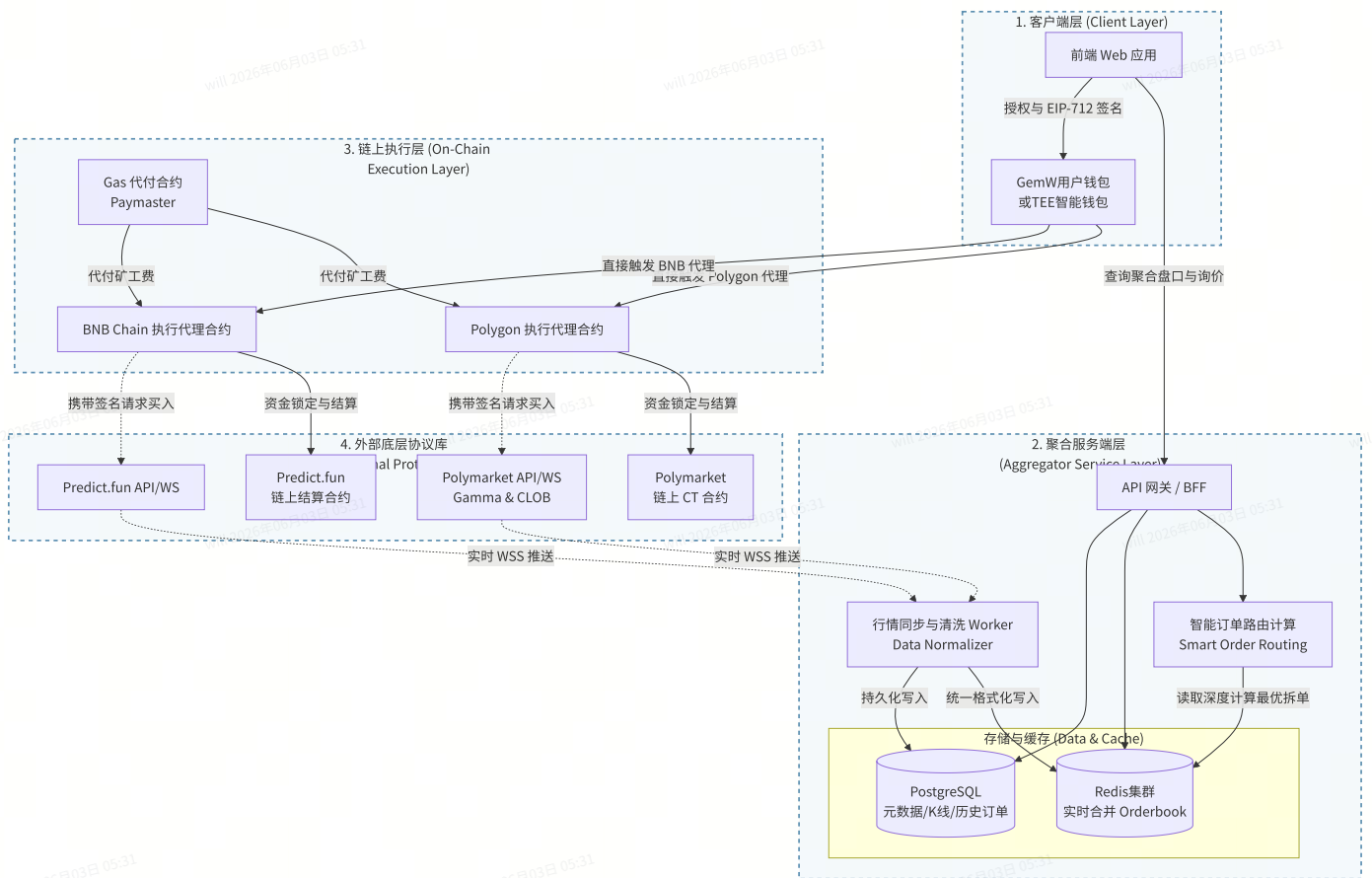
- 初版聚合器上线，时间一个半月，7月底
- 自研预测市场上线（预言机开发），时间一个月，8月底
- 无风险套利做市商MM上线，时间半个月，9月中旬

0x1 背景

当前的预测市场流动性相对割裂，用户为了寻找最优赔率往往需要在多个平台间横跳，聚合器能完美解决这个痛点。

根据当前市场占有率，初期聚合Polymarket（Polygon链），Predict.fun（BNB Chain链）2家市场流动性设计该方案。

0x2 核心架构设计



1. 各链的**执行代理合约**为入口合约，用于实现**平台手续费抽水**，**分佣**
2. TEE智能钱包（邮箱登录钱包）可以实现各链自动化**资金rebalance**

三个核心模块：数据接入、聚合引擎 和 交易路由。

1. 数据同步与标准化层 (Data Ingestion & Normalization)

两个平台的 API 数据结构完全不同，需要一个中间层来抹平差异。

- 静态数据同步 (REST API)：定时拉取 Polymarket (Gamma API) 和 Predict.fun 的市场元数据（市场 ID、标题、分类、结束时间）。
- 实时盘口同步 (WebSocket)：建立长连接，监听两个平台的 Orderbook 变化和最新成交价。
- 数据清洗 (Normalization)：将两边的数据结构统一为标准 JSON 格式。例如，统一将 Yes/No 转化为标准的二元期权模型，并对齐标的物（如把两边关于“2024美国大选”的相同事件强行绑定在**同一个聚合 ID**下）。

2. 聚合引擎层 (Aggregation Engine)

后端的性能核心，需要处理高并发读写。

- 统一订单簿 (Unified Orderbook): 将两边的深度合并, 按价格从优到劣排序, 计算出聚合后的全网最优买价 (Best Bid) 和最优卖价 (Best Ask)。
- 实时缓存 (In-Memory Cache): 订单簿数据绝不能直接读写关系型数据库, 必须全部放在 Redis 等内存数据库中, 以毫秒级响应前端查询。

3. 交易路由与执行层 (Execution Router)

当用户要在聚合器上下单时, 系统需要决定去哪边吃单。

- 询价与路由 (Smart Routing): 根据用户输入的购买金额, 计算在 Polymarket 还是 Predict.fun 下单滑点更低, 或者是否需要拆单 (Split Order) 两边同时买入。
- 签名与上链 (EIP-712): 由于预测市场的交易大都是链下订单簿匹配 + 链上结算, 用户用自己的钱包对聚合器生成的 Payload 进行签名, 然后由后端代为推送到对应的 CLOB (中央限价订单簿) API。

0x3 事件对齐 (Event Mapping) - 用AI做

同一个预测事件, Polymarket 叫 "Trump wins 2024", Predict.fun 可能叫 "Will Donald Trump be elected in 2024?". 用 LLM (如 OpenAI API) 做语义相似度匹配来自动对齐事件, 然后引入人工审核机制, 否则一旦对齐错误, 交易路由就会引发灾难。

第一步: 向量检索初筛 (Vector Search) —— “找相似”

需要将两个平台所有的事件标题、描述转化为向量, 存入向量数据库 (如 Milvus 或 Pinecone), 为 Polymarket 的每个事件找出 Predict.fun 中相似度最高的 Top 5 候选。

- 模型 1: `text-embedding-3-small` (OpenAI) —— 极度便宜, 速度快, 多语言支持好, 足以应对大部分英文标题。
- 模型 2: `bge-m3` (智源 BAAI 开源) —— 把向量化过程本地部署省钱, 目前最强的开源多语言文本表征模型。

第二步: LLM 精准裁决 (Reasoning & Decision) —— “做判断”

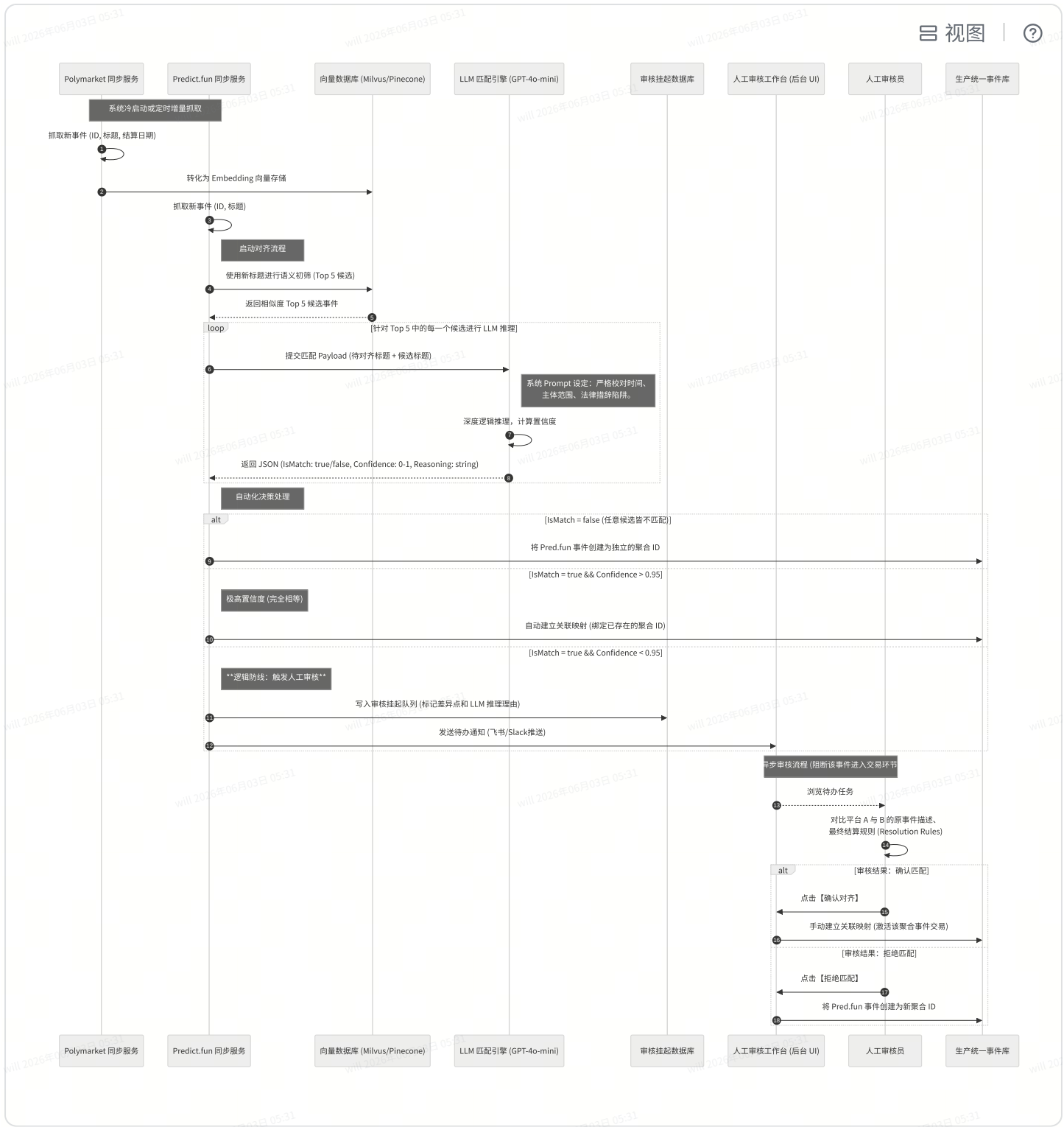
拿到 Top 5 候选后, 把 Polymarket 的原事件和这 5 个候选事件丢给 LLM, 让它判断哪个是 **完全等价** 的标的, 并输出严格的 JSON。

在模型的选择上, 需要平衡逻辑推理能力和成本:

模型推荐	适用场景	优势与核心理由
GPT-4o-mini (OpenAI)	首选：日常高频对齐	性价比之王。它的指令遵循能力极强，输出标准 JSON 格式极度稳定，且成本极低的直白预测事件（如体育比分、主流选举）完全够用。
Claude 3.5 Sonnet (Anthropic)	备选：复杂歧义事件	逻辑推理天花板。对于充满法律术语、结算条件极其苛刻的事件（比如“SEC 是通过某 ETF”），Claude 抓取细微语义差异的能力目前强于 GPT-4o。
Qwen 2.5 (72B) / Llama 3.1 (70B)	私有化部署	如果你的服务器已经具备 GPU 算力，不想被 API 频控卡脖子，这两个开源模型通后，配合精准的 Prompt，能力完全能对标 GPT-4。

事件对齐主要核心流程是：





0x4 智能订单路由引擎 (Smart Order Routing - SOR)

场景：当用户输入“我要花 100000 USDC 买川普胜选”时，Polymarket只能承载80000USDC深度，另外20000USDC需要拆到Predict去执行。

后端引擎必须在一瞬间完成最优拆单计算。这不仅仅是比对两边的盘口价格，还要把跨链损耗和 Gas 算进去。

最优化算法来最小化用户的总成本。目标函数可以抽象为：

$$\min \left(\sum_{i \in \{Poly, Pred\}} \int_0^{Q_i} P_i(q) dq \right) + Fee_{gas} + Fee_{bridge}$$

- Q_i : 在单个平台分配的购买数量，满足约束 $Q_{Poly} + Q_{Pred} = Q_{Total}$ 。
- $P_i(q)$: 平台 i 订单簿中吃单深度对应的价格函数（考虑滑点）。
- Fee_{bridge} : 如果需要跨链，跨链桥产生的固定费用和流动性折损。

路由逻辑：

1. 读取 Redis 中 Polymarket 和 Predict.fun 的实时合并订单簿。
 2. 计算如果全部在本地链 (如 Polygon) 吃单的平均价。
 3. 计算拆分一半去 BNB Chain，加上跨链桥手续费后的综合成本。
 4. 如果拆单综合成本更低，则生成**跨链拆单执行方案**。
- 接入 **基于意图的快速桥 (Intent-based Bridge)**，如 **deBridge (DLN)** 或 **Across Protocol**，它们通过做市商垫资，能将跨链时间从几分钟缩短到 10 秒以内

0x5 API 速率限制 (Rate Limits)

两个平台的公共 API 都有严格的频控。生产环境中，需要准备多个 API Key 轮询

0x6 自动化资金处理

Polymarket 使用 Polygon 上的 USDC 结算，而 Predict.fun 在 BNB Chain 上使用 USDT 结算。

- GemW

前端页面提示用户自动换取标的资产，会产生一定闪兑手续费。

- Web3 钱包-跨链资金结算

如果用户钱包里只有 Polygon USDC，但他想买的聚合最优价格在 Predict.fun，集成跨链桥（如 Across 或 Stargate）帮用户完成跨链，显示跨链手续费。

0x7 合规与风控

触碰了用户的资产路由，需要限制敏感地区（如美国）的 IP 直接访问交易功能。

